

An Intro to Kubernetes



By Alex Bissessur

Recap of Containers

- Originates from BSD with chroot, later Jails
- Project created by a madman on drugs
- Jails was used to run binaries in an isolated environment
 - no access to main filesystem
 - no ability to make changes to host OS

BSD's Jail

Jails: Confining the omnipotent root.

The FreeBSD “Jail” facility provides the ability to partition the operating system environment, while maintaining the simplicity of the UNIX “root” model. In Jail, users with privilege find that the scope of their requests is limited to the jail, allowing system administrators to delegate management capabilities for each virtual machine environment. Creating virtual machines in this manner has many potential uses; the most popular thus far has been for providing virtual machine services in Internet Service Provider environments.

Containers Today

- Docker is the leading containerisation standard
RedHat has Podman because RedHat is *different*TM
- An encoding of deployment procedures in images
<https://github.com/cloud-native-mauritius/cloudnativemauritius.com/>
- Images are reproducible and portable between machines (and OS-es)
- The developer becomes *the operator*.

Virtual Machines

- Before VM OS, there were individual machines for isolation
- Virtual Machines are heavy
- Every VM needs the kernel to be emulated
10 Vms => 10 kernels
- Operating systems don't do nothing
- Leads to expensive resource bloat

How Containers Help

- Uniform development environment
- Dependency and package conflicts avoided
- Security risks do not spread to other applications
- Easier control over environment your apps run in

How Containers Help

- Containers use the host kernel
Less wasted resources
- No 2nd OS (hypervisor) to run
- Integration with CI/CD pipelines
- Prevent over-provisioning of resources
- Facilitates microservices architecture
- Opens the door to Kubernetes & Cloud

The Development Process @ SWAN

- I write a .NET application
- I build it for prod and wait 2 minutes
- Go to the folder, manually edit a config file to make the app run on IIS
- Virtual Desktop to the server
- Copy my files over in the kbps speeds
- (re)Start the service

And it doesn't work.

The Development Process @ SWAN

- Write application in Python
- Hans clones project & tries to run it. Fails.
- Install/update dependencies
- Conflicting dependencies for different programs
- Inconsistencies between my laptop & Linux server

And the deadline's approaching.

- Note to self
Run ubuntu container
Then run Top

Don't include this slide in pres

How Does Kubernetes Fit In?

What Is Cloud Native?

- The term for systems built for the needs of the 21st century
- Mindset prioritising:
 - High Availability (**no** downtime)
 - Scalability (match user demand)
 -
- No, Cloud Native doesn't mean just Cloud (AWS, GCP, Azure)

What Is Kubernetes?

- “K8S is a system for automating deployment, scaling and management of containerised applications”
 - Simply an abstraction/framework to run a cluster of computers
- Industry standard with a whole ecosystem behind
- Platform for 200~ Cloud Native Applications
- Former Google product, now Open Source held by CNCF (Linux Foundation project)

Why is Kubernetes a Thing?

- Need for cluster computing solutions which don't suck (Docker Swarm 🤢)
- High Availability, Redundancy, Easy to Manage
- CSP agnostic (AWS / Azure / GCP / Bare Metal)
- Offers a platform suited to both admins and devs

Why Use Kubernetes

- Single PC finitely upgradeable
- Expand horizontally not vertically
Quantity over quality
- Redundancy
- Exploiting microservices and containers
- Better upgradeability, availability, portability

Building on Kubernetes

- The power of Kubernetes is the CN Landscape
- ~ 200 Cloud Native projects and counting

- CI/CD
- App Definition & Image Build
- Databases
- Streaming & Messaging
- Scheduling & Orchestration
- API Gateway
- Discovery
- Cloud Native Storage
- Cloud Native Network
- Service Mesh
- Remote Procedure Call
- Service Proxy
- Chaos Engineering
- Continuous Optimisation
- Container Runtime
- Security & Compliance
- Container Registry
- Automation & Configuration
- Key Management
- Observability
- Feature Flagging
- Coordination & Service

Physical Scaling

- One server cannot be upgraded indefinitely
- Physical limitations in compute power
- Instead of 1 server, why not have 10?
Kubernetes unifies multiple servers
- Kubernetes spreads pods across all nodes to best use resources

Virtual Scaling

- Scaling microservices
Possible due to lack of state
- Allows optimal use of resources
- Scale up/down to accomodate demand
- Horizontal Pod Autoscaler for hands-off scaling
- Automatic load balancing of applications

Redundancy

- 1 server = 1 point of failure
- Self-healing of deployments in case of failure
 - Pods independent from nodes
 - Services “follow” pods
- Longhorn/Rook+Ceph allows distributed block storage for data redundancy
- Multi-cloud cluster for added redundancy
- Benefits physical upgrades or OS updates

Saving the Developer

- Focus on code, not deployments.
- Automatic deployments & updates regardless of if Hans is online. (CI/CD)
- Allocate resources to applications.
- Environment consistency.
- Microservices
- GitOps – Version Control

Application on localhost



Status 200
Success Response

Same code when deployed



Cors Error
Cookies Sharing issues
Bad Request

Fancy Deployment Strategies

- Deployments can have different rollout strategies
- Algorithms like *Canary* deployment, *Rolling* update
- Blue/Green deployment concept
- Detailed control over rollout to minimise customer impact

Demo Time



Kubernetes offers several benefits to developers:

1. **Simplified Deployment:** Automates the deployment process, making it easier to release applications consistently.
2. **Scalability:** Enables developers to scale applications up or down effortlessly based on demand.
3. **Resource Management:** Automatically allocates resources, allowing developers to focus on code rather than infrastructure.
4. **Environment Consistency:** Provides consistent environments from development to production, reducing "it works on my machine" issues.
5. **Microservices Support:** Facilitates building and managing microservices architectures, enabling independent development and deployment.
6. **CI/CD Integration:** Easily integrates with continuous integration and continuous deployment (CI/CD) pipelines, streamlining development workflows.
7. **Self-Healing:** Automatically restarts failed containers, ensuring higher availability and reliability of applications.
8. **Version Control:** Supports versioning of applications, making rollbacks and updates more manageable.